AI Museum of the Future

Technical report (D1)

Multi-agent curation of a virtual exhibition for a visitor in 2126

Team project — ESPRIT / Deep Learning
Focus period: Tunisia 2020–2026
Draft based on repository measurements (October 2026)

Abstract

This report describes an AI system that builds a virtual museum exhibition about our era as if curated one hundred years in the future. Public contemporary sources are collected, indexed with retrieval-augmented generation (RAG), analysed by five specialist agents (historian, sociologist, technology, culture & art, visual), peer-reviewed, synthesised by a curator into four to six sections, and fact-checked. Image understanding supplies the mandatory multimodal component. The pipeline is exposed through a Flask API, a web Exposition page, and an n8n end-to-end workflow. Design choices for chunking and retrieval are measured; multimodal analysis exposes captions that contradict the image content.

Keywords: multi-agent systems, RAG, multimodal vision, n8n, museum curation, Tunisia 2020–2026.

Contents

1 Problem and concept **2**

2 Architecture **2**

3 Data and RAG (B5) **3**
3.1 Corpus . . . 3
3.2 Chunking . . . 3
3.3 Retrieval . . . 3

4 Multimodal image understanding (C4) **4**

5 Agents and prompt engineering **5**

6 Orchestration, web application and n8n **5**
6.1 HTTP API (`exhibition_api.py`) . . . 5
6.2 Exposition page . . . 5
6.3 n8n . . . 6

7 Evaluation **6**

8 Limits and next steps **6**

1. Problem and concept

Central question. *If a museum were created in 100 years to represent our time, what would it preserve, and how would it tell our story?*

The system answers that question for a controlled period and place (default: **Tunisia, 2020–2026**). It does not predict future historians with certainty, crawl the entire Internet, train a foundation model, or build a full 3D/VR museum (out of scope for the five-week project).

Why several agents instead of one prompt. A single model asked to "write an exhibition" tends to invent details, blur perspectives, and hide disagreement. Specialists examine the *same* evidence under distinct roles; the peer reviewer makes agreements and tensions explicit; the curator must cite chunk IDs; the fact-checker marks unsupported claims. Multi-agent design therefore improves transparency and grounding, not only style. It also matches the pedagogical goal of the course: separable, testable components (data, RAG, vision, prompts, automation) rather than one opaque generation step.

Input / processing / output.

- **Input:** news and events, images, science, technology, social and cultural trends (public APIs).
- **Processing:** cleaning, RAG retrieval, multimodal image analysis, specialised agents, cross-agent synthesis, curatorial selection.
- **Output:** exhibition title and concept, 4–6 sections with narratives, selected artefacts with citations, visual presentation on a web page.

Scope (Week 1 brief). In scope: controlled corpus, 4–5 specialists plus curator, RAG, image understanding, automated n8n workflow, final virtual exhibition with several sections. Out of scope: crawling the whole web, certainty about future historians, full VR museum, training a foundation model from scratch, unlimited data sources.

2. Architecture

1. **Public data layer** — collectors for GDELT, Wikipedia/Wikidata, World Bank, arXiv, Wikimedia, Openverse, Met, Internet Archive, DBpedia, plus Tunisian full text.
2. **Knowledge layer** — sentence chunking, local multilingual embeddings (384 dimensions), Supabase Postgres + pgvector (HNSW) and full-text search.
3. **Multimodal layer** — vision LLM describes source images; descriptions are cached, logged, stored in document metadata and made searchable.
4. **Specialised agents** — five perspectives in parallel, each with its own retrieval queries.
5. **Review, curation, fact-check** — peer review; curator (4–6 sections, retries); fact-checker (section support + optional rewrite).
6. **Delivery** — every run is saved under `runs/<run_id>/run.json`; Flask API and Exposition UI; n8n orchestration.

Models. Embeddings: `paraphrase-multilingual-MiniLM-L12-v2` (local, free). Chat/vision: OpenAI-compatible provider (default Groq, e.g. `gpt-oss-120b` / vision-capable model). Configuration lives in `agents/llm_config.py`.

Run log. Each execution writes `runs/<YYYYMMDD-HHMMSS-xxxxxx>/run.json` with events (`evidence`, `insight`, `abstain`, `images`, `review`, `result`, ...) and timings. A run can be scored offline (`evaluation.evaluate_run`) or replayed (`from_run`: curator + fact-check only, without re-calling the five specialists). This makes demos and debugging reproducible when the LLM quota is empty.

Information flow (logical). Collectors fill `documents`/`chunks`; each specialist retrieves with its own queries; the Visual agent adds `vision:` lines; the reviewer compares insights; the curator emits sections whose artefacts are chunk IDs; the API resolves those IDs to titles, URLs and `image_url` for the Exposition page.

3. Data and RAG (B5)

3.1. Corpus

Sources are official public APIs only. An audit (2026-10-14) showed a knowledge base dominated by titles (median document length 68 characters) and only about 75 chunks mentioning Tunisia. Remediation included 92 Tunisian Wikipedia articles (EN/FR/AR, ≈1.9 M characters) and full text for hundreds of news/archive items. After re-ingestion, chunks mentioning Tunisia rose from 75 to 3,747 (Latin) and from 6 to 1,417 (Arabic); total chunks from 7,296 to 16,994.

3.2. Chunking

The embedding model reads only its first **128 tokens**. With former 512-token chunks, 57% of the text was invisible to vector search. Five strategies were compared on 23 passage-level questions (EN/FR/AR):

Strategy	Unseen text	hit@5	MRR@10
fixed-512 (former)	57%	0.52	0.40
recursive-600	6%	0.70	0.43
sentence-128 + title (chosen)	**0%**	0.65	**0.47**

Table 1: Chunking comparison (summary). Full table in `rag/README.md`.

Choice: whole sentences packed to 128 tokens of the model's tokenizer, one-sentence overlap, document title as prefix.

3.3. Retrieval

Hybrid vector + Postgres full-text with reciprocal-rank fusion, year/country filters, statistics down-weighted, one chunk per document. Measured on 16 fixed queries, k=5 (2026-10-15):
LLM reranking lowers quality and costs ≈24 s per query; it is **off by default**. Indexing: HNSW (recall@10 = 1.00 vs 0.67 for the former ivfflat setting on the measured sample). Full-text GIN index cut median keyword latency from hundreds of milliseconds to tens of milliseconds.

Method	precision@5	hit@5	MRR
vector	0.50	0.88	0.61
hybrid (default)	**0.70**	**1.00**	**0.94**
hybrid + LLM rerank	0.56	1.00	0.75

Table 2: Retrieval after new corpus and HNSW + full-text indexes.

Agent-side retrieval. Beyond a shared candidate pool, each specialist searches with domain queries. The sociologist may use HyDE (Hypothetical Document Embeddings). Filters prefer in-country evidence but can relax when the country filter is too sparse, marking out-of-scope hits. Duplicate arXiv/news rows from parallel Python and n8n ingestion were removed with `rag.dedupe_documents`.

What to write for B5 in the oral. Justify: (1) public APIs only; (2) 128-token window ⇒ sentence packing; (3) hybrid over pure vector; (4) rerank off after a full measurement; (5) HNSW over ivfflat. Cite the numbers in the tables above.

4. Multimodal image understanding (C4)

Image understanding is the project's mandatory non-textual modality. The Visual agent does not invent pictures: it selects real source images and asks a vision model what they show.

Pipeline.

1. Download with a descriptive User-Agent (Wikimedia returns HTTP 403 without one).
2. Vision LLM returns structured JSON (`objects`, `scene`, `description`, `tags`, `caption_mismatch`).
3. Results are disk-cached, appended to evidence as `vision: ...`, recorded on the run's `images` event, written to `documents.metadata.vision_*`, and appended as a searchable `Vision:` line on the chunk.
4. Hybrid retrieval surfaces `vision_*` fields so every agent can use what the pixels show.

Measured contribution. Caption: "Protest in Mexico City". Vision: shirtless men in swim trunks on a road, cars, billboards—**no protest** (`caption_mismatch`). Text-only RAG would trust the caption; multimodality exposes the contradiction. Similar mismatches appear when an economy article is illustrated with a banknote from another country, or when a climate label is only readable in the artwork itself. The Exposition page shows source images and "Ce que l'IA voit".

Limits of the multimodal stack. We embed the *textual description* of the image in the same 384-d space as document chunks (not a separate CLIP image tower in pgvector). That is enough to search by visual content via keywords and vectors of the description, and stays compatible with the existing schema without a mass re-ingest. Only images actually analysed by the Visual agent (bounded by `VISUAL_MAX_IMAGES`) are written back to Supabase.

Agent	Role	Notes
Historian	Socio-historical framing	Prompt versioned in `prompts/`
Sociologist	Society / civic dynamics	HyDE on own search
Technology	Tech / innovation (Tasnim)	Prompt v5, measured iterations
Culture & Art	Cultural production	Country-scoped evidence
Visual	Image-led artefacts	Up to `VISUAL_MAX_IMAGES` visions/run
Reviewer	Agreements / tensions / weak findings	Shared
Curator	4–6 sections, cited narrative	Shared
Fact-checker	Section support vs evidence	Shared

Table 3: Specialists and synthesis agents.

5. Agents and prompt engineering

Technology agent (Tasnim). Techniques include role and scope, strict JSON, cited findings only, a finding budget (3–5 findings, at most 2 interpretations), and few-shot examples taken from live failures (e.g. inventing technology from a headline, guessing a place name). Iterations v3→v4→v5 were measured on real Supabase evidence: invented technologies and guessed names went to zero in the v4 samples; v5 hedges interpretations ("may suggest") and forbids outcomes no source measures. A separate HTTP demo exists (`serve_technology.py`, port 5002) and an n8n template. Details: `docs/technology/PROMPT_LIBRARY_TECHNOLOGY.md`.

Shared design. Every specialist searches the knowledge base with its own queries; findings without valid `[chunk_id]` citations are dropped; with nothing left, the agent abstains and appears in `missing_perspectives`. The curator may list missing perspectives explicitly so the exhibition does not silently invent a voice that failed.

Prompt library. Prompts are versioned under `prompts/` with change logs; the dashboard validation step lists the nine system/agent files and their current versions. Earlier iterations and before/after JSON dumps live under `evaluation/prompt_iterations/`.

6. Orchestration, web application and n8n

6.1. HTTP API (`exhibition_api.py`)

- `POST /api/exhibition` — start a full run, or `from_run` to replay curator + fact-check.
- `POST /api/exhibition/<run_id>/cancel` — cooperative stop at the next step.
- `POST /api/exhibition/<run_id>/vision` — describe remaining exhibition images.
- `GET /api/exhibition/<run_id>`, `/latest`, `GET /api/exhibitions`.

One run at a time (HTTP 409 otherwise). Optional `EXHIBITION_API_TOKEN`. Errors returned to clients are generic (provider details stay in the server log).

6.2. Exposition page

Route `http://127.0.0.1:5000/#exhibition`: launch, follow, stop, load saved runs, replay curator, gallery of artefacts, fact-check badges, peer review and per-agent status.

6.3. n8n

Workflow `n8n/workflows/Exhibition_pipeline_end_to_end.json`: webhook or weekly schedule → optional ingest → start run → poll every 60 s → summary or failure. Use `127.0.0.1` rather than `localhost` on Windows (IPv6 vs IPv4). The embedding path for the separate chunking workflow also calls `POST /api/embed` so n8n vectors stay comparable to Python ingestion. Documentation: `n8n/README.md`.

Demo path (for D5). (1) Show dashboard validation. (2) Open Exposition, load a saved Tunisia run. (3) Point to citations and images. (4) Open "Ce que l'IA voit" on a mismatched caption. (5) Optionally trigger n8n webhook or "Rejouer le curateur". Keep a precomputed run ready when the free LLM quota is exhausted.

7. Evaluation

- **Chunking / retrieval:** Section 3 tables (scripts under `evaluation/`).
- **Saved runs:** `python -m evaluation.evaluate_run runs/<run_id>` (coverage, citation survival, fact-check support, timings). Optional `-judge` for faithfulness (needs LLM quota).
- **Automated tests:** offline `pytest` suite (API, multimodal, RAG, agents). Live sociologist tests may be deselected when the free LLM quota is empty.

Reproduce.

```
pip install -r requirements.txt
cp .env.example .env          # own keys only
python -m pytest -q --deselect \
  tests/test_sociologist.py::TestSociologistLive
python app.py                 # :5000/#exhibition
```

8. Limits and next steps

- **E1 — Provider limits.** Free-tier tokens-per-minute can abort the curator rewrite after fact-check; the exhibition is kept without the rewrite.
- **E2 — Strict fact-check.** Often only a minority of sections are fully supported; the curator may need several attempts to reach 4–6 sections.
- **E3/E4 — Quality of specialist claims** still depends on each agent's prompt and evidence.
- Tunisian precision remains below global hybrid precision; more local sources remain valuable.
- **D5** — 15-minute demo script; optional AI-generated museum-hall illustration (separate image credits).

Team principle. Roles (data, RAG, multi-agent, multimodal, prompts/evaluation, n8n) are responsibility areas, not silos: every member should understand the full pipeline.

Ownership of this report draft. Sections 2, 4 (implementation), 5 (Technology), and 6 are primarily filled from Tasnim's branch work (exhibition API/UI, multimodal persistence, n8n end-to-end, Technology prompts). Sections 3 and 5 for other agents should be completed by the RAG owner and each specialist owner before the final submission. Section 7 should paste metrics from one agreed complete run.

This PDF is generated from `docs/report/REPORT_D1.tex`. *Source outline:* `docs/REPORT_D1.md`.

Appendix A — File map

- `exhibition_api.py` — HTTP exhibition API (start, cancel, vision, list).
- `web/exhibition.js`, `web/exhibition.css` — Exposition UI.
- `multimodal/analyze_image.py`, `store_vision.py` — vision + C3 persistence.
- `n8n/workflows/Exhibition_pipeline_end_to_end.json` — automation.
- `run_exhibition.py` — CLI full pipeline / `-from-run`.
- `evaluation/evaluate_retrieval.py`, `compare_chunking.py`, `evaluate_run.py`.

Appendix B — Example API body

```
POST /api/exhibition
{
  "country": "Tunisia",
  "year_from": 2020,
  "year_to": 2026,
  "review": true,
  "fact_check": true
}
```

Replay curator only:

```
{ "from_run": "20261015-113815-09c5e1", "fact_check": true }
```

Appendix C — Multimodal checklist (C1–C3)

1. **C1 — Analyze:** download with a browser-like User-Agent; reject non-image MIME types; call Groq vision with a structured JSON schema (caption, objects, scene, text_in_image, confidence).
2. **C2 — Cache:** disk cache keyed by image URL/hash so re-runs and peer review do not re-bill the vision API.
3. **C3 — Persist:** write `vision_*` fields into `documents.metadata`; prepend a searchable `Vision:` line into chunk content so hybrid retrieval can surface visual context even when the Wikimedia caption is wrong or incomplete.

Mexico-style mismatch. When the caption says one thing and the pixels show another, the Exposition UI exposes "Ce que l'IA voit" so the visitor (and the fact-checker) can compare caption vs. vision description.

Appendix D — Suggested run metrics to paste for Section 7

Before the final PDF submission, replace placeholders with one agreed complete Tunisia run:

- Run id, wall-clock duration, specialist timings.
- Number of sections after curator (target 4–6).
- Fact-check supported / unsupported counts.
- Citation survival (sections still citing retrieved chunks).
- Number of images with vision metadata shown in Exposition.

References

[1] AI Museum of the Future — Week 1 brief (use case, scope, roles, n8n, architecture).

[2] Project file `rag/README.md` — measured chunking and retrieval results.

[3] Project file `multimodal/README.md` — vision pipeline and Mexico caption case.

[4] `docs/technology/PROMPT_LIBRARY_TECHNOLOGY.md` — Technology prompt iterations.